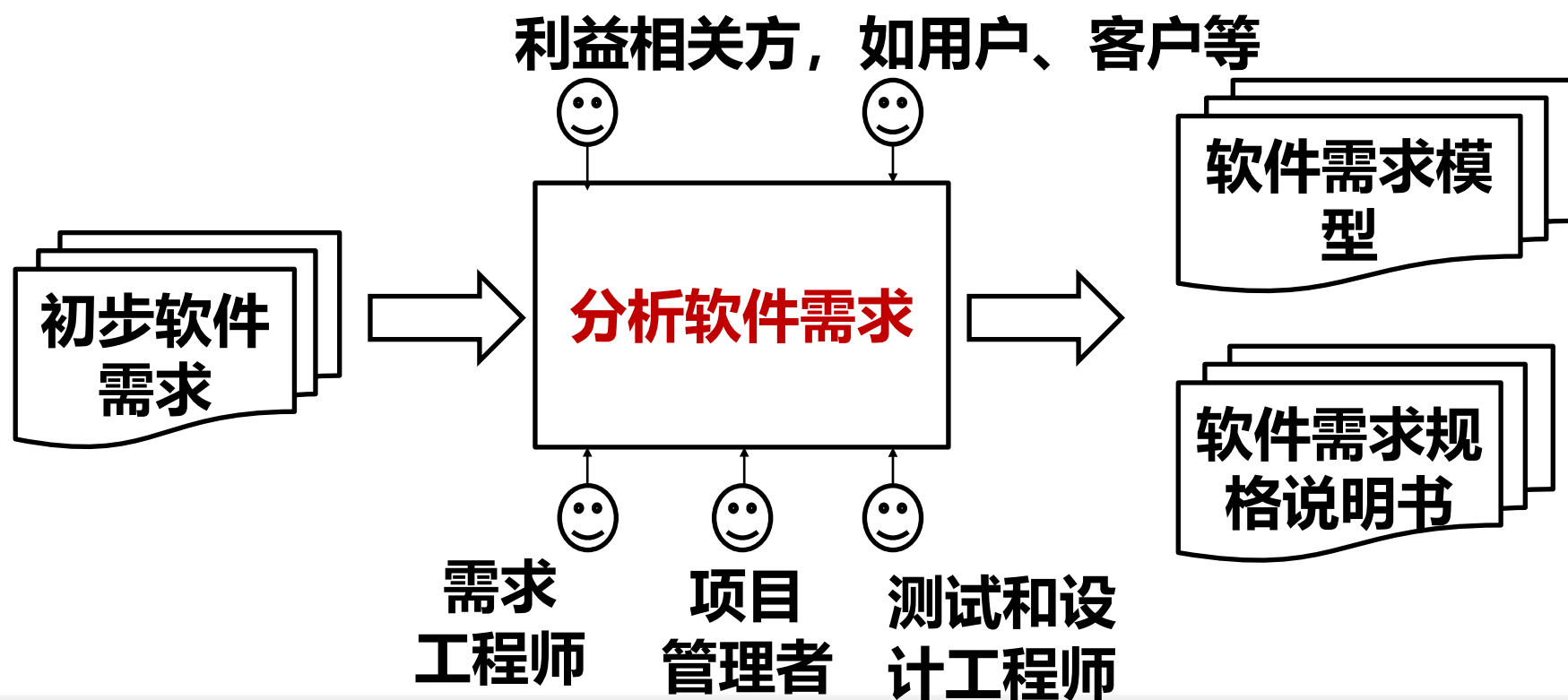


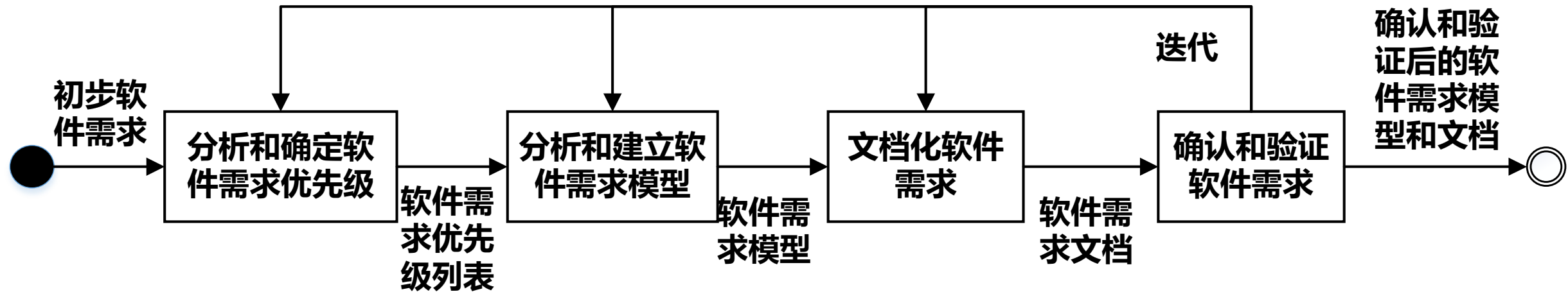
# 软件需求分析

# 1 软件需求分析的任务

□ 基于初步软件需求，进一步**精化和分析**软件需求，确定软件需求**优先级**，建立软件**需求模型**，发现和解决软件需求**缺陷**，形成高质量的软件需求模型和软件**需求规格说明书**



# 1.1 需求分析过程



## 1.2 分析和建立软件需求模型的具体步骤

分析和建立  
用例交互模型

分析和确定用例  
所涉及的对象类

分析和确定对  
象间消息传递

绘制用例  
的交互图

交互模型

分析和建立  
分析类模型

确定分  
析类

确定分析类  
职责和属性

确定类  
间关系

绘制分  
析类图

类模型

分析和建立  
状态模型

绘制状态图

# 1.3 软件需求的不同视角表示

## □用例视角

- ✓ 具有哪些**功能**、功能间有何**关系**、功能与利益相关方有何**关系**
- ✓ UML提供了**用例图**来分析和描述用例视角的软件需求模型

## □行为视角

- ✓ 用例是如何通过业务领域中一组对象以及它们间的**交互**来达成的
- ✓ UML提供了**交互图**、**状态图**来描述行为视角的软件需求模型

## □结构视角

- ✓ 业务领域有哪些重要的领域**概念**以及它们之间具有什么样的**关系**
- ✓ UML提供了**类图**来描述和分析业务领域的概念模型

## 2.1 分析和建立用例的交互模型

### □任务

- ✓分析和描述用例是如何通过一组对象之间的交互来完成的

### □步骤

- ① 分析和确定用例所涉及的对象及其类
- ② 分析和确定对象之间的消息传递
- ③ 绘制用例的交互图

# 分析和确定用例所涉及的对象及其类

□ 软件需求用例的处理通常涉及三种不同类对象

- ✓ 边界类
- ✓ 控制类
- ✓ 实体类

□ 这些类是在用例分析阶段所识别并产生的，通常将它们称为分析类

# 边界类

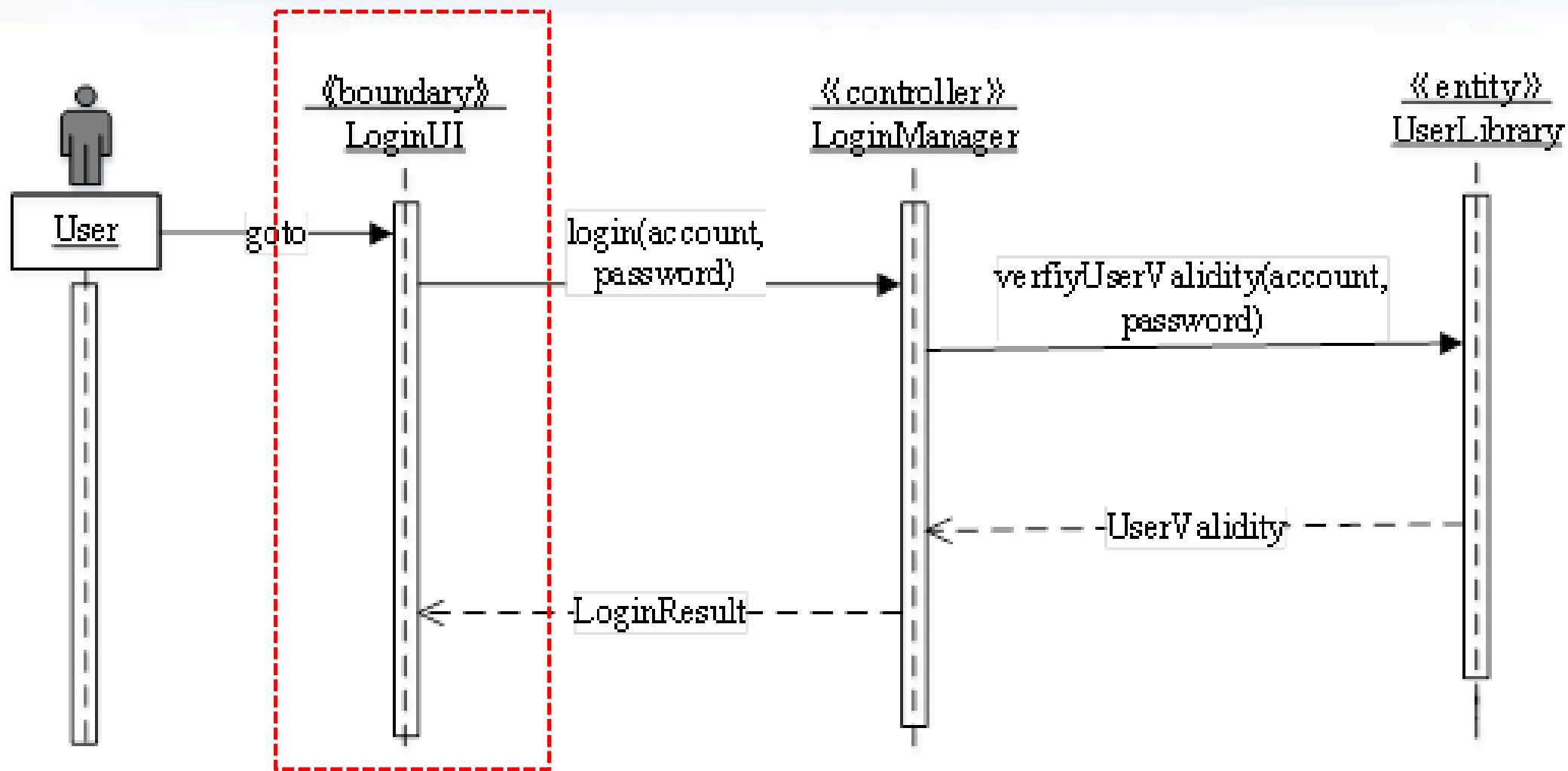
## □何为边界类

- ✓每个用例或者由外部执行者触发，或者需要与外部执行者进行某种信息交互，因而用例的业务逻辑处理需要有一个类对象来负责目标软件系统与外部执行者之间的交互
- ✓由于这些类对象**处于系统的边界**，需与系统外的执行者进行交互，因而将这些对象所对应的类称之为边界类

## □边界类对象主要起到以下作用

- ✓交互控制，处理外部执行者的输入数据，或者向外部执行者输出数据
- ✓外部接口，如果外部执行者表现为其他的系统或者设备，那么边界类对象需要与系统之外的其他系统或设备进行信息交互

# 示例：边界类



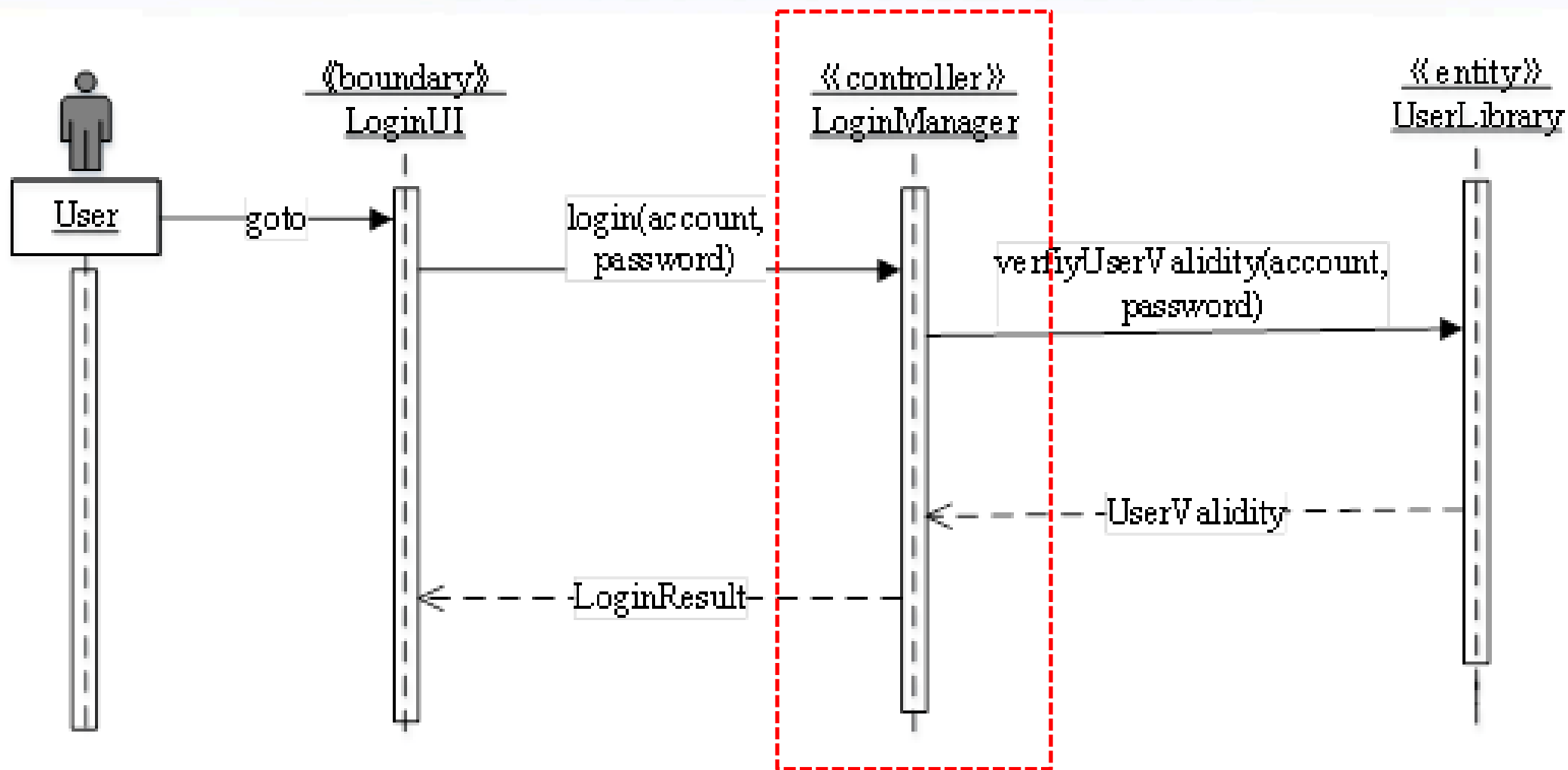
# 控制类

## □控制类对象作为完成用例任务的主要协调者

- ✓负责处理边界类对象发来的任务请求，对任务进行适当的分解，并与系统中的其他对象进行协同，以控制他们共同完成用例规定的任务或行为

## □一般而言，控制类并不负责处理具体的任务细节，而是负责分解任务，并通过消息传递将任务分派给其他对象类来完成，协调这些对象之间的信息交互

# 示例：控制类

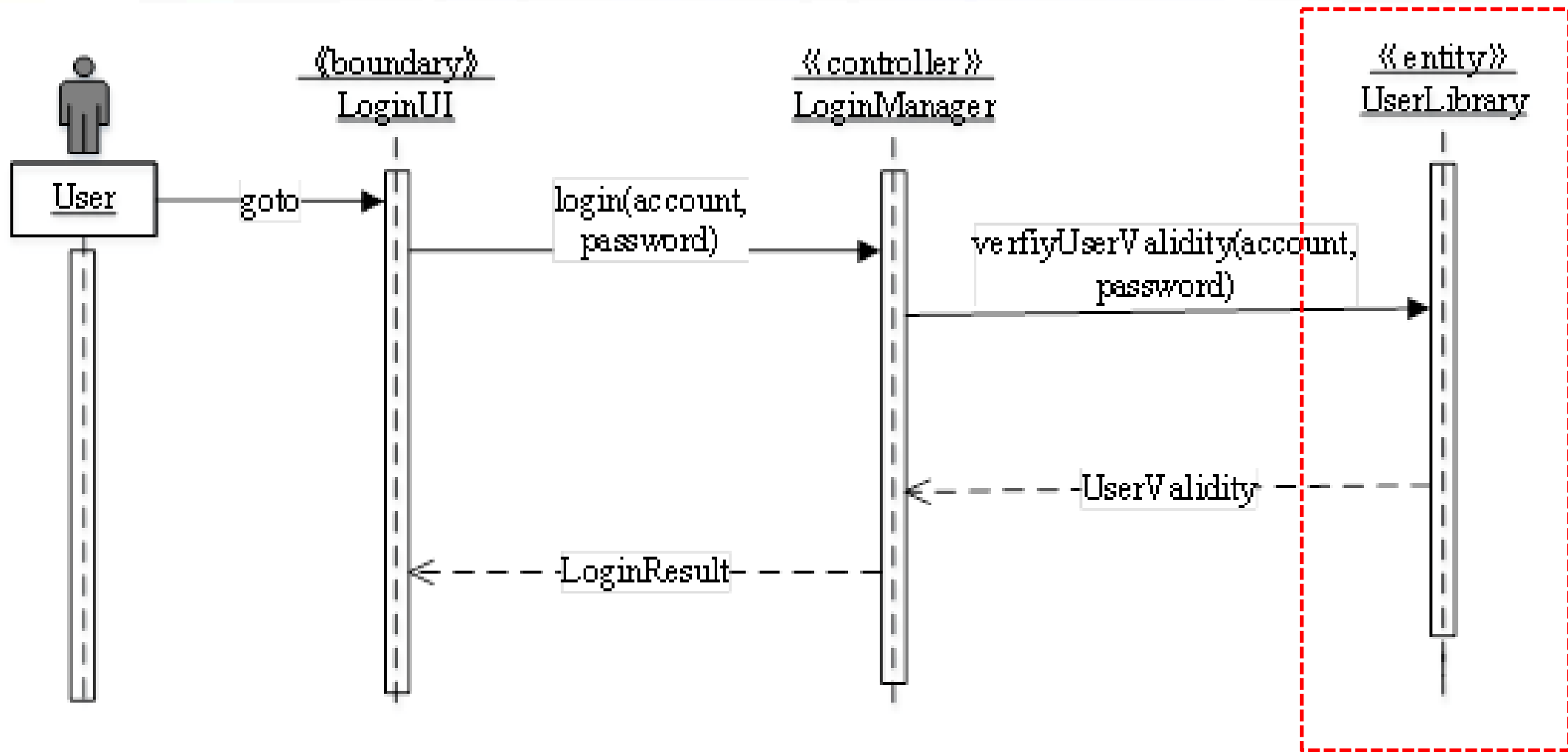


# 实体类

□用例所对应**业务流程中的所有具体功能**最终要交由具体的类对象来完成，这些类称之为**实体类**

- ✓一般地，实体类对象负责保存目标软件系统中具有持久意义的信息项，对这些信息进行相关的处理（如查询、修改、保存等），并向其他类提供信息访问的服务
- ✓实体类的职责是要落实目标软件系统中的用例功能，提供相应的业务服务

# 示例：实体类



## 2.2 分析和确定对象之间的消息传递

- 确定消息的名称

- 确定消息传递的信息

# 确定消息的名称

## □消息的名称

- ✓直接反映了对象间**交互的意图**，也体现了接收方对象所对应的类需**承担的职责和任务**，也即发送方对象希望接收方对象提供什么样的功能和服务
- ✓**意图**：请求、通知

## □消息名称的表示

- ✓一般地，消息名称用**动名词**来表示
- ✓需求工程师应尽可能用**应用领域中通俗易懂的术语**来表达消息的名称和参数，以便于用户和需求工程师等能直观地理解对象间的交互语义

# 确定消息传递的信息

□对象间的交互除了要表达消息名称和交互意图之外，在许多场合还需要提供必要的**交互信息**

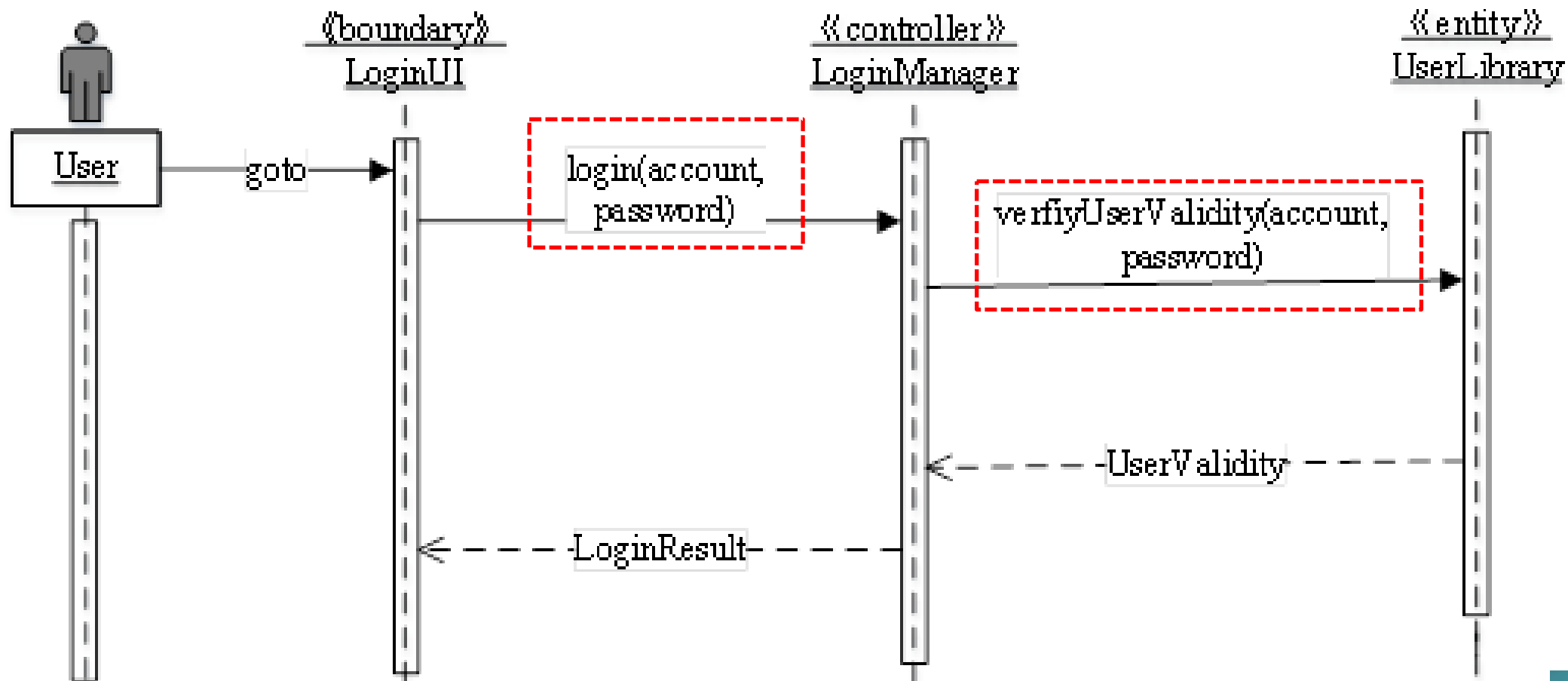
✓这些信息通常以消息参数的形式出现，也即一个对象在向另一个对象发送消息的过程中，需要提供必要的参数，以向目标对象提供相应的信息

✓**信息**：通知和请求的内容

□在构建用例的交互图过程中，如果用例的业务流程能够明确相应的交互信息，那么就需要确定消息需附带的信息

□通常，消息参数用名词或名词短语来表示

# 示例：消息的名称和信息



消息的意图和信息是什么？



# 绘制用例的交互图

## □绘制顺序图的策略

- ✓用例的外部执行者应位于图的最左侧，紧邻其右的是用户界面或外部接口的边界类对象，再往右是控制类对象，控制类的右侧应放置实体类对象，它们的右侧是作为外部接口的边界类对象
- ✓对象间的消息传递采用**自上而下的布局方式**，以反映消息交互的时序先后

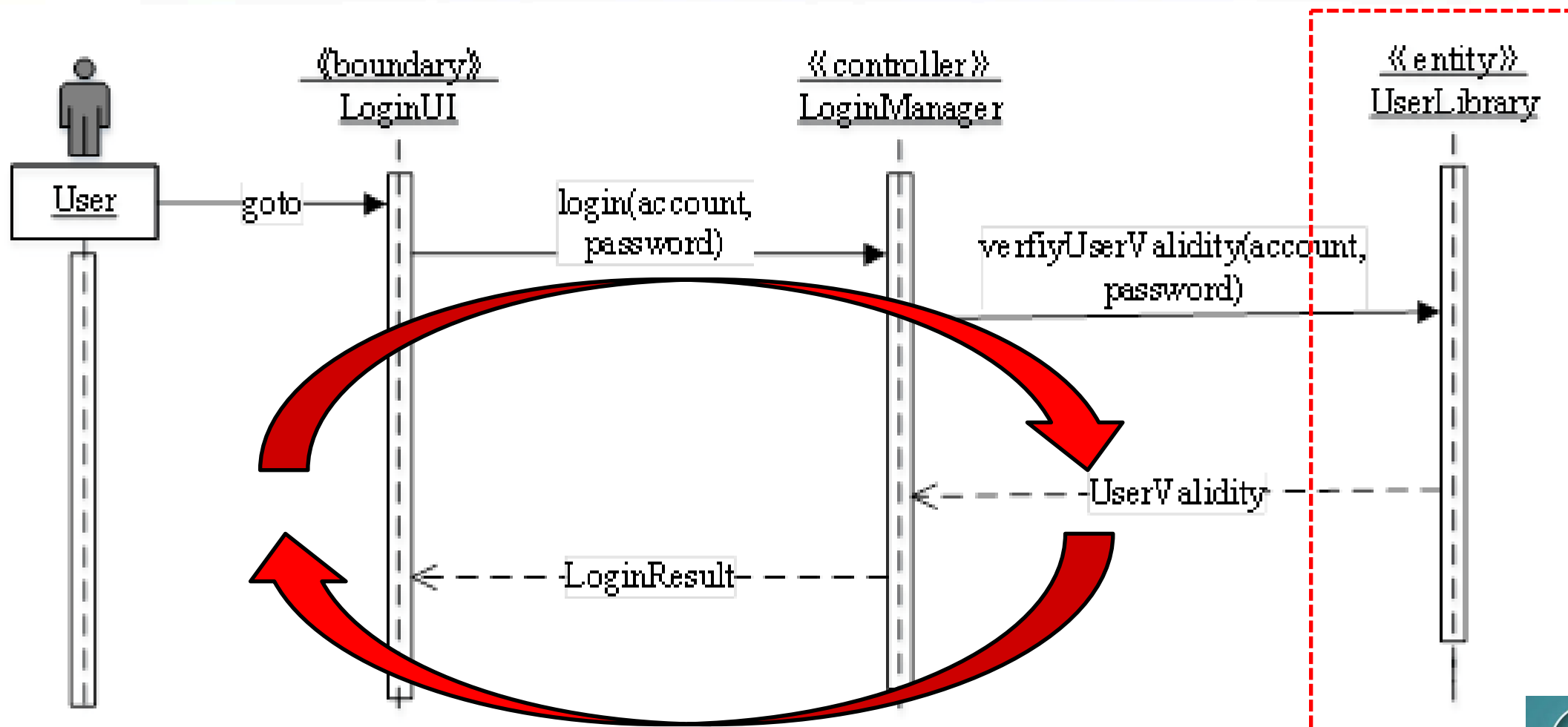
## □一个用例至少构造一个交互图，以刻画用例的行为模型

- ✓对于较为简单的用例，为其构造一个交互图就足够了
- ✓对于较复杂的用例而言，需要为此用例绘制多张交互图，每张交互图刻画了用例在某种特定场景下的交互动作序列

# 用例交互图的工作流程

- ① 外部执行者与边界类对象进行交互以启动用例的执行
- ② 边界类对象接收外部执行者提供的信息，将信息从外部形式转换为内部形式，并通过消息传递将相关信息发送给控制类对象
- ③ 控制类对象根据业务逻辑处理流程，产生和分解任务，与相关的实体类对象进行交互以请求完成相关的任务，或者向实体类对象提供业务信息，或者请求实体类对象持久保存业务逻辑信息，或者请求获得相关的业务信息
- ④ 实体类对象实施相关的行为后，向控制类对象反馈信息处理结果
- ⑤ 控制类对象处理接收到的信息，将处理结果通知边界类对象
- ⑥ 边界类对象对接收到的处理结果信息进行必要的分析，将其从内部形式转换为外部形式，通过界面将处理结果展示给外部执行者

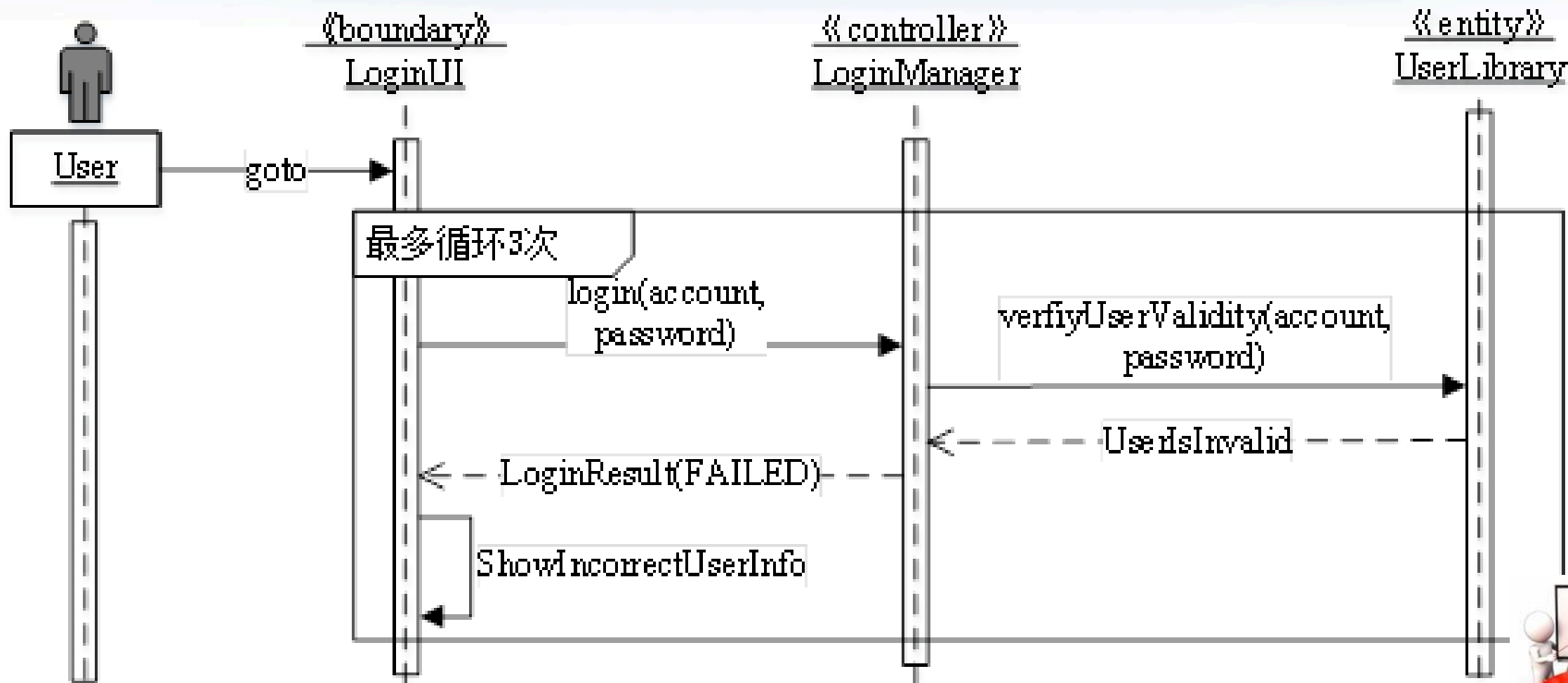
# 示例：顺序图的工作流程



顺序图的工作流程有何特点？



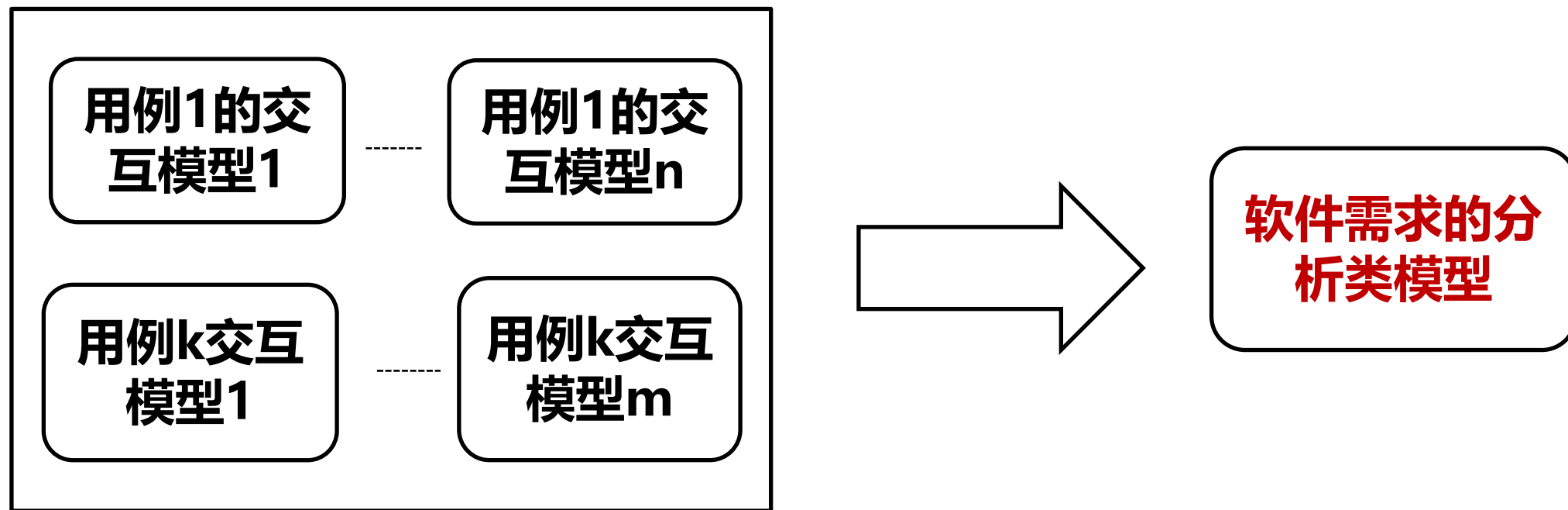
# 思考和讨论



一个用例用一张顺序图描述够吗？为什么？



## 2.2 分析和建立软件需求的分析类模型



# 分析和建立分析类模型的步骤

- ① 确定分析类
- ② 确定分析类的职责
- ③ 确定分析类的属性
- ④ 确定分析类之间的关系
- ⑤ 绘制分析类图

## 2.2.1 确定分析类

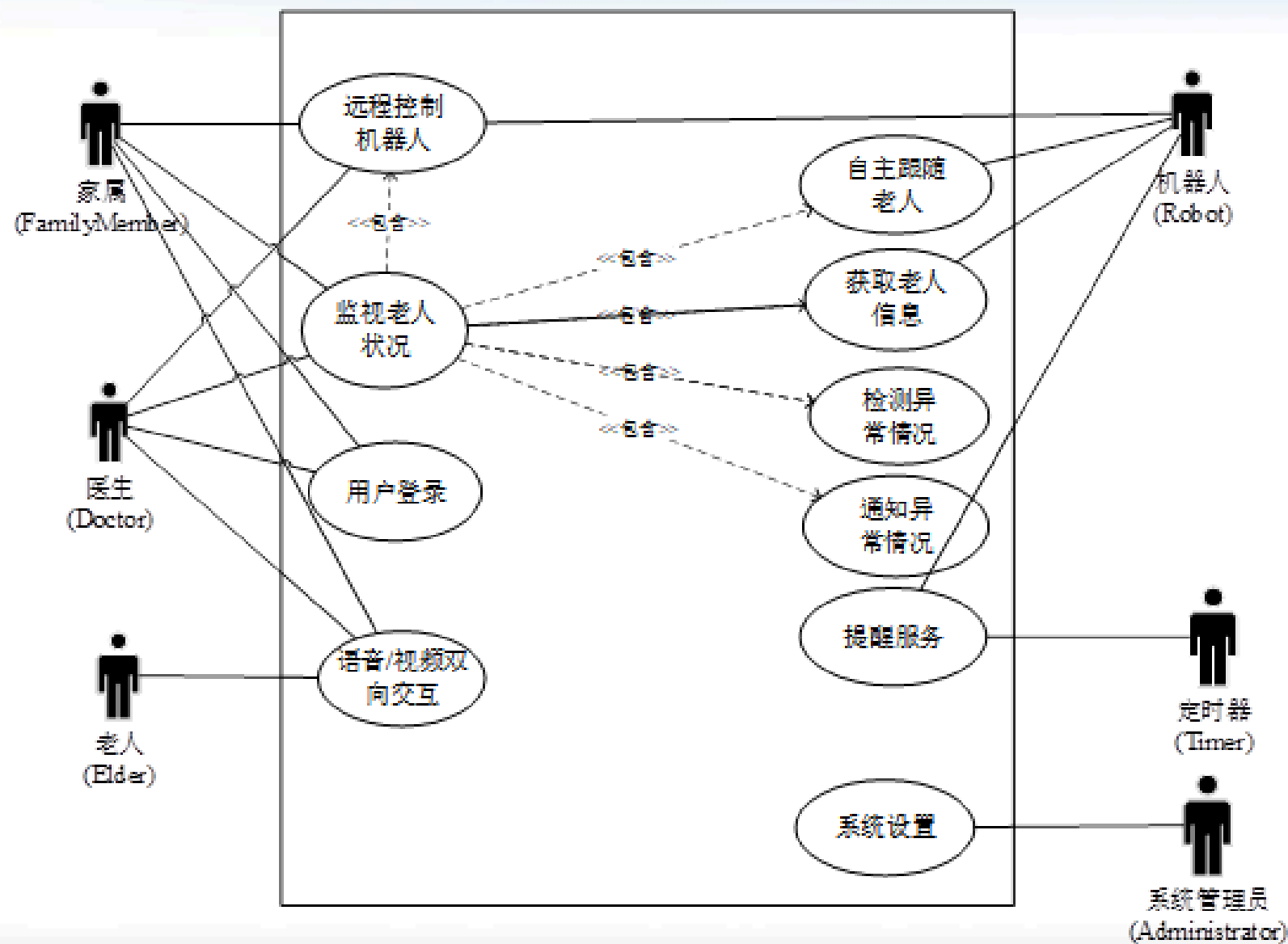
□有哪几种分析类？



□如何确定分析类？

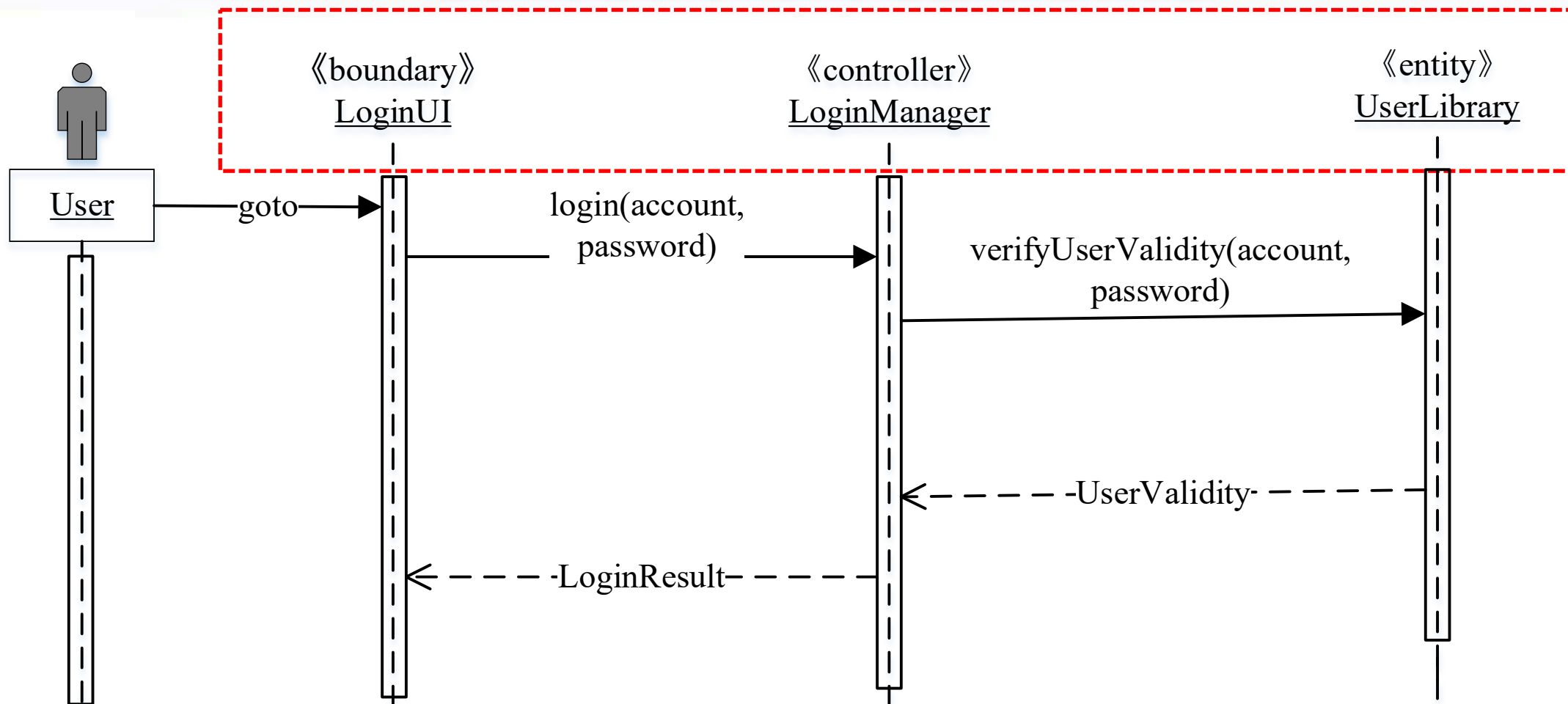
- ✓用例模型中的**外部执行者**应该是分析类图中的类
- ✓在各个用例的顺序图，如果该图中出现了某个**对象**，那么该**对象所对应的类**属于分析类，应出现在分析类图中

# 示例：根据用例图来确定分析类



有哪些分析类？

# 示例：根据交互图来确定分析类

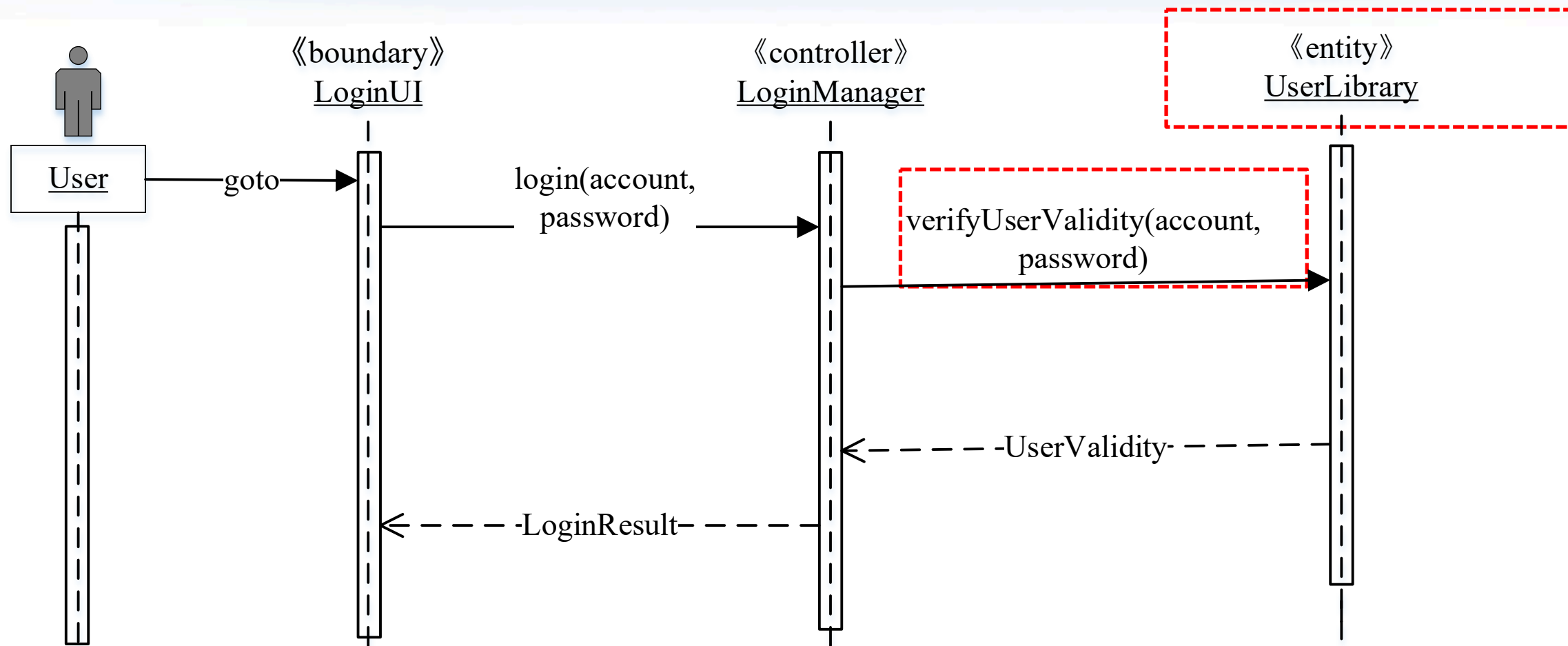


有哪些分析类？

## 2.2.2 确定分析类的职责

- 每一个分析类都有其**职责**，需提供相关的**服务**
- 对象接收的**消息与其承担的职责**之间存在一一对应关系，  
即如果一个对象能够接收某项消息，它就应当承担与该消息相对应的职责
- 可用类的**方法名**来表示分析类的职责，并采用简短的自然语言来详细刻画类的职责

# 示例：确定分析类的职责

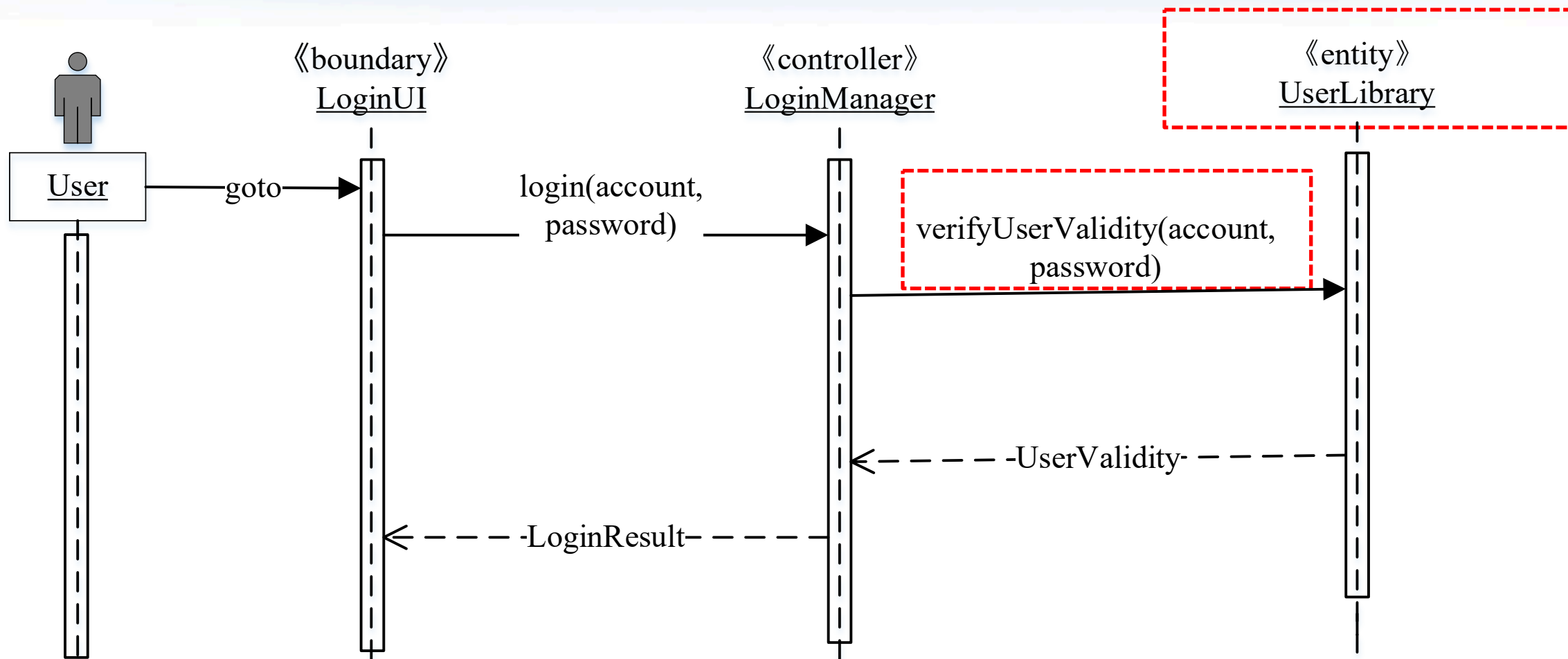


**UserLibrary类有何职责？**

## 2.2.3 确定分析类的属性

- 分析类具有哪些属性取决于该类需要持久保存哪些信息
- 用例顺序图中的每个对象所发送和接收的消息中往往附带有相关的参数，这意味着发送或接收对象所对应的类可能需要保存和处理与消息参数相对应的信息，因而可能需要与此相对应的属性

# 示例：确定分析类的属性



**UserLibrary类有何属性?**

## 2.2.4 确定分析类之间的关系

□从全局视角理解和描述不同分析类间的关系

□类之间的关系有多种形式

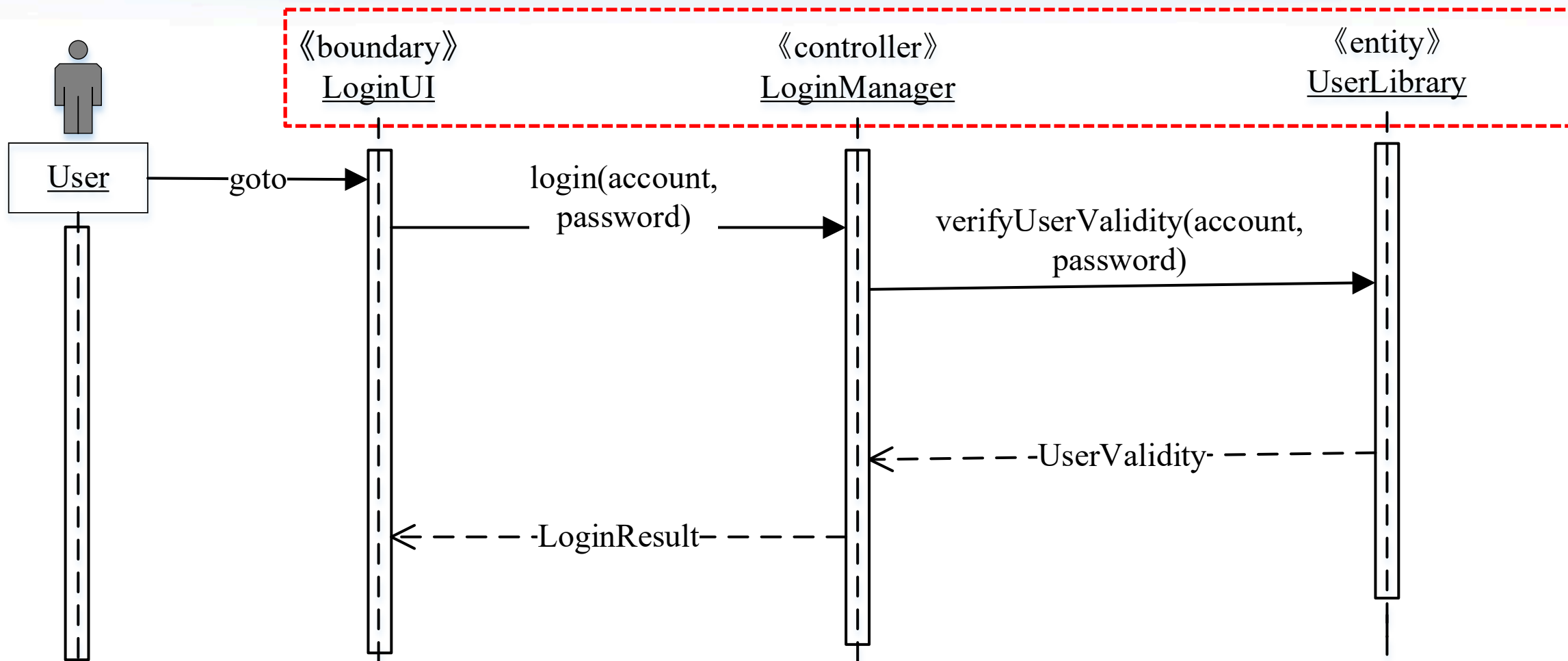
✓包括继承、关联、聚合和组合、依赖等等

□具体策略

✓在用例的顺序图中，如果存在从类A对象到类B对象的消息传递，那么意味着类A和B间存在**关联、依赖、聚合或组合**等关系

✓如果经过上述步骤所得到的若干个类之间存在一般和特殊的关系，那么可对这些分析类进行层次化组织，标识出它们间的**继承关系**

# 示例：确定分析类之间的关系



哪些分析类之间存在怎样的关系？

## 2.2.5 绘制分析类图

### □制出系统的分析类图，建立分析类模型

- ✓直观描述了系统中的分析类、每个分析类的属性和职责、不同分析类之间的关系

### □如果系列规模较大，分析类的数量多，关系复杂，难以用一张类图来完整和清晰地表示，那么可以分多个子系统来绘制分析类图

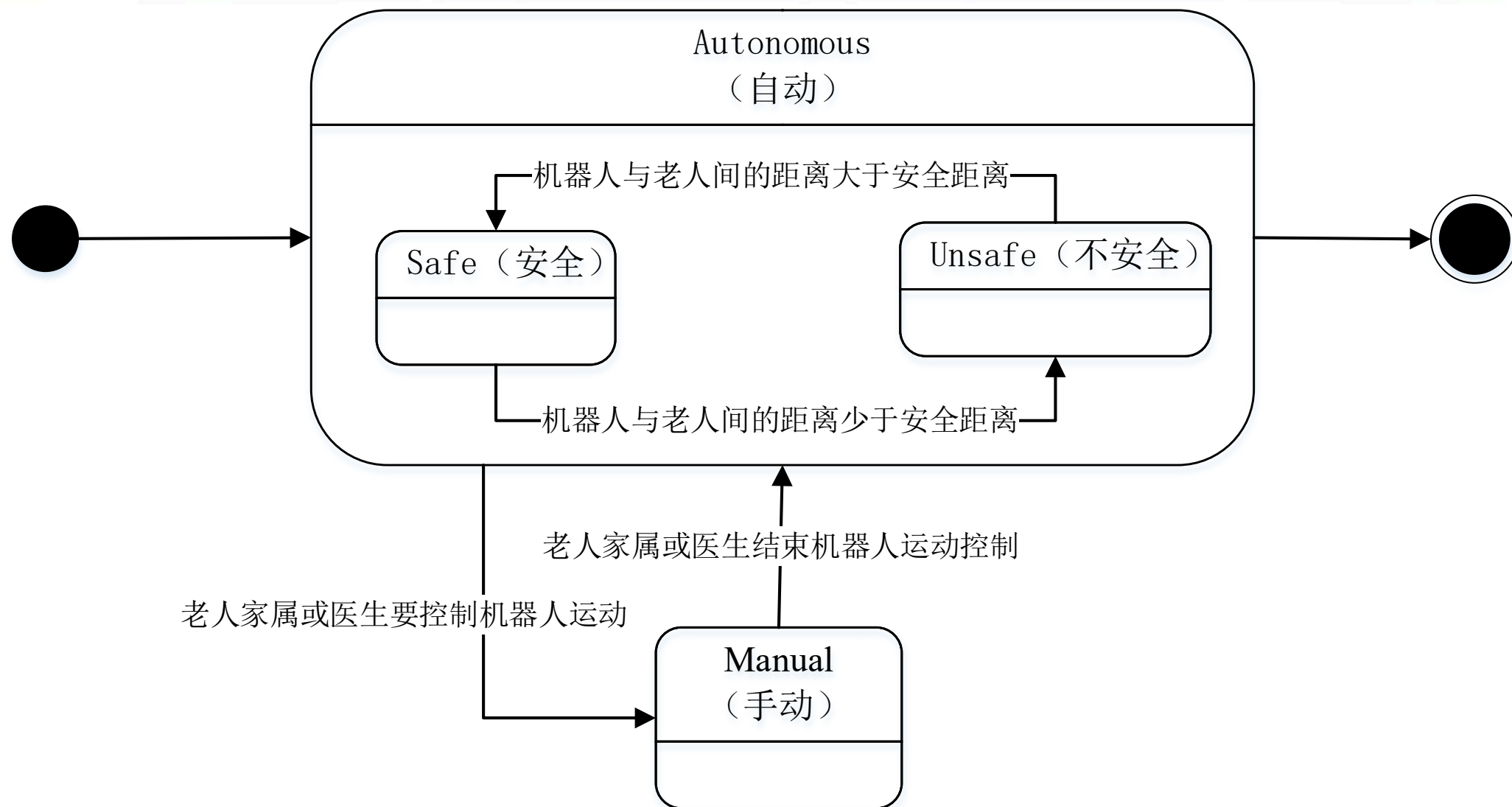
## 2.3 分析和建立软件需求的状态模型

□用UML的状态图来描述这些对象的**状态模型**，以刻画对象拥有哪些状态、对象的状态如何受事件的影响而发生变化

### □注意二点

- ✓状态模型是针对**对象**而言的，而非针对分析类
- ✓需求工程师**无需为所有**的类对象建立状态模型，只需针对那些具有复杂状态的对象建立状态模型

# 示例: 分析机器人人类对象的状态图



## 2.3.1 状态图

### □功效

- ✓描述实体（对象、系统）在**事件**刺激下的**反应式动态行为**及其导致的**状态变化**
- ✓刻画了实体的可能状态、每个状态下可响应事件、响应动作、状态迁移

### □图的构成

- ✓**节点**：状态
- ✓**边**：迁移，即状态间因事件刺激而触发的状态变化

# 状态图的基本概念

□ **状态**: **对象属性取值**构成的一个约束条件

✓ 不同状态下对象对事件的响应行为完全一样

□ **事件**: **某时刻点发生、需要关注的瞬时刺激或触动**

✓ **消息型事件(同步)**: 其他对象发来消息

✓ **信号型事件(异步)**: 其他对象传来异步信号

✓ **时间型事件**: 到达特定时间点

✓ **条件型事件**: 对象属性取值满足特定条件

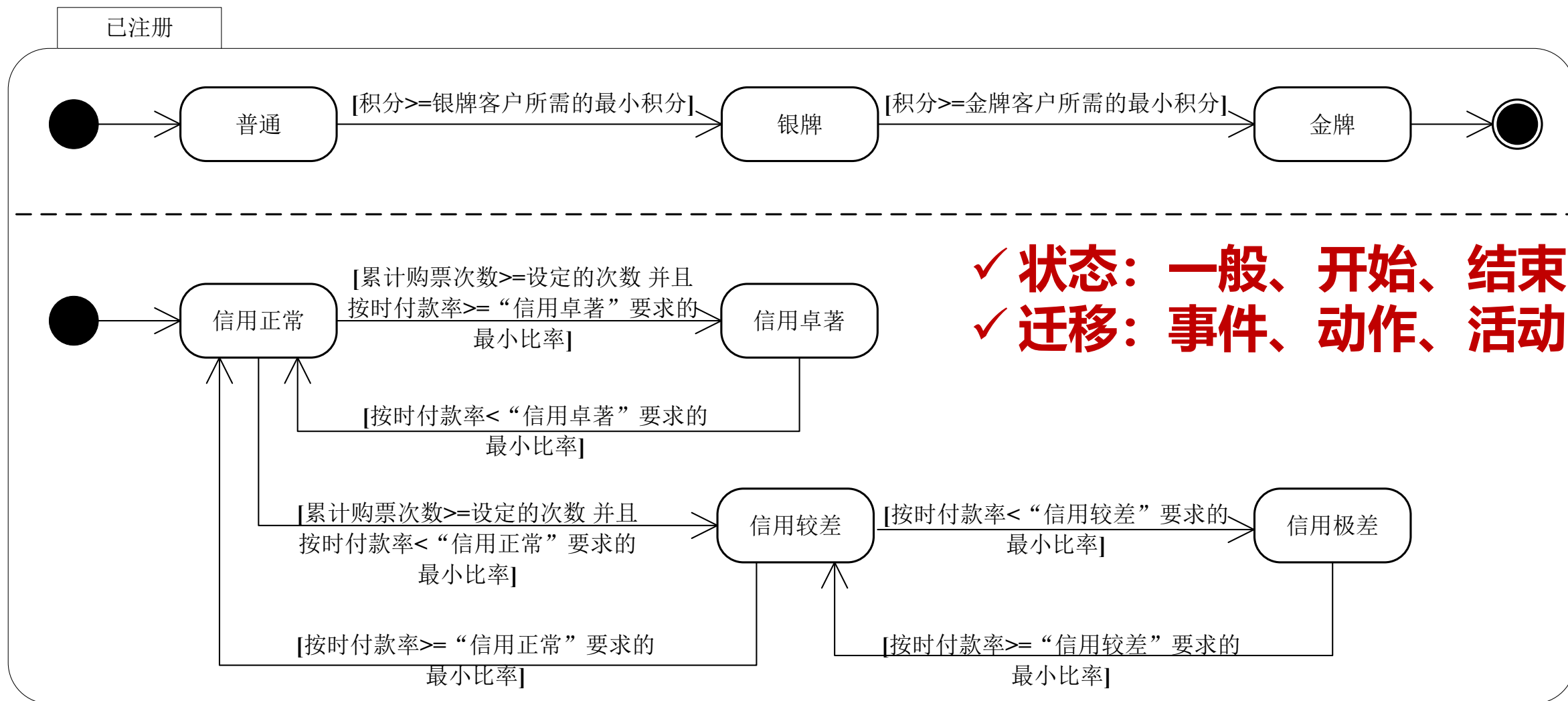
□ **动作(action)**

✓ 计算过程, **位于迁移边上**, 简单, 执行时间短

□ **活动(activity)**

✓ 计算过程, **位于状态中**, 复杂、执行时间长

# 示例：状态图



# 复合状态

- 状态 $s$ 是由子状态 $s_1, \dots, s_n$ 经**AND（与）逻辑复合**而成  
是指，对象处于状态 $s$ 当且仅当对象同时处于 $s_1, \dots, s_n$
- 状态 $s$ 由子状态 $s_1, \dots, s_n$ 经**XOR（异或）逻辑复合**而成  
是指，对象处于状态 $s$ 当且仅当对象处于 $s_1, \dots, s_n$ 中的某个子状态

# 状态的表示

## □由状态名及可选的入口活动、出口活动、do活动、内部迁移构成

- ✓一旦对象经迁移边从其他状态进入本状态，那么本状态**入口活动**将被执行
- ✓一旦对象经迁移边从本状态进入其他状态，那么本状态的**出口活动**将被执行
- ✓**do活动**是当对象进入本状态并执行完入口活动（如果有的话）后应该执行的活动。
- ✓**内部迁移**不会引起对象状态的变化，所以它虽有源状态（包含此迁移的状态），但没有目标状态

# 迁移的表示

## □迁移标注为“[事件][监护条件][ / 动作]”

- ✓“**事件**”表示触发此次状态变迁的事件
- ✓“**监护条件**”表示约束状态迁移真正发生的条件表达式
- ✓“**动作**”表示状态迁移期间应当执行的动作 (action)
- ✓在监护条件和动作中均可引用触发事件参数和对象属性

## □迁移表示为状态节点间的有向边

- ✓自迁移是指源状态与目标状态相同的特殊外部迁移

# 状态图绘制原则

## □绘图

- ✓状态节点按行上下对齐，按列左右对齐
- ✓迁移边绘制成水平线段、垂直线段或者由水平、垂直两种线段组合而成的折线
- ✓避免斜线

□初态应位于左上角，终态尽量靠近右下角

□迁移边上的文本标注应靠近源状态

## 2.3.2 UML交互图的作用

### □刻画对象间的消息传递，分析如何通过交互协作完成功能

- ✓用例的功能实现方式
- ✓软件系统在某种使用场景下对象间的交互协作流程
- ✓软件系统的某个复杂操作的逻辑实现模型

### □二类交互图

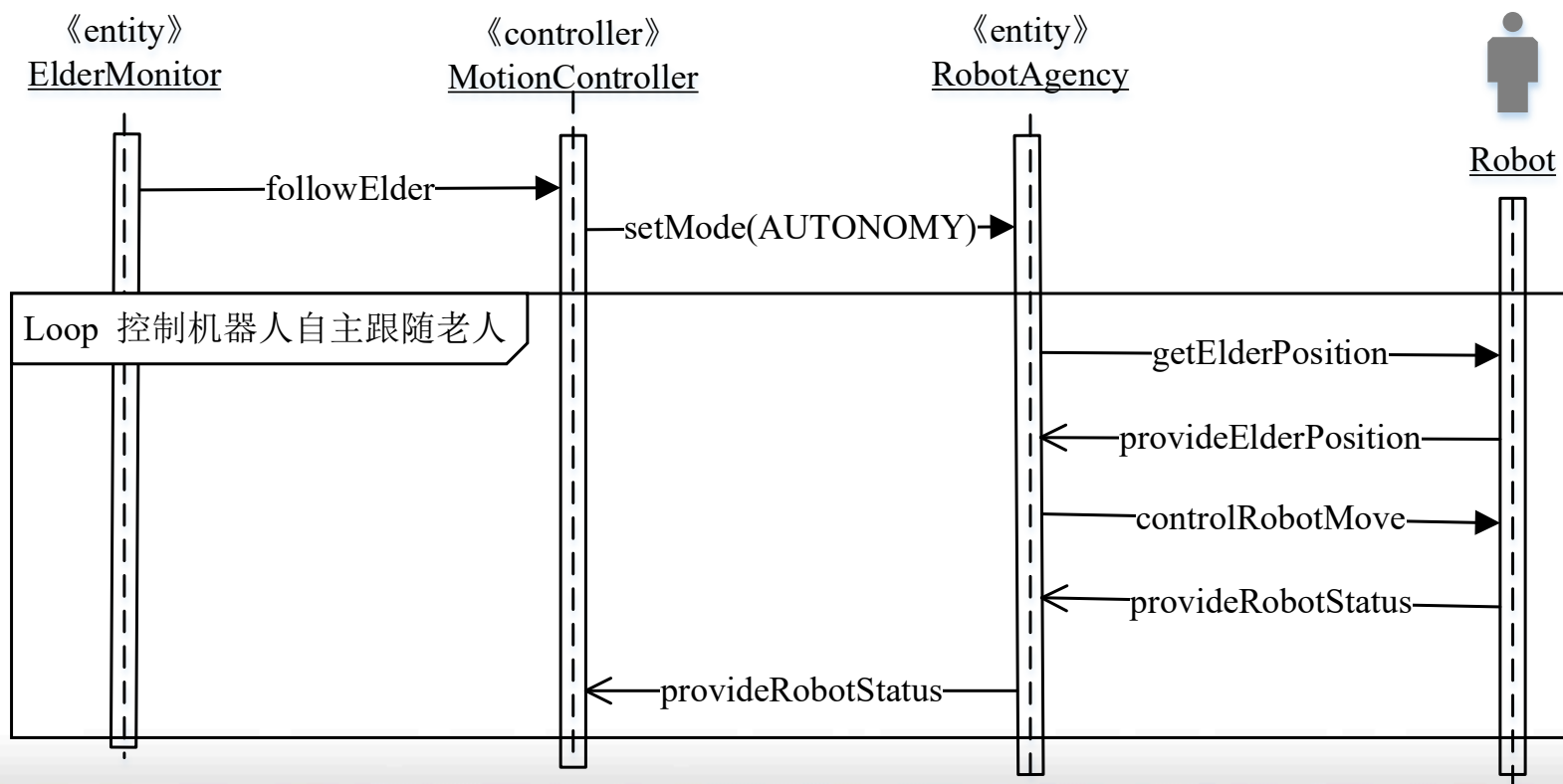
- ✓顺序图(Sequence Diagram): 强调消息传递的时间序
- ✓通信图(Communication Diagram): 突出对象间的合作

### □表达能力相同，二类图可相互转换

# 顺序图

## □描述对象间的消息交互序列

- ✓ **纵向**：时间轴，对象及其生命线(虚线)，活跃期(长条矩形)
- ✓ **横向**：对象间的消息传递



# 顺序图的表示方式

## □对象

- ✓ “[对象名] : [类名]”
- ✓ 示例: “Tom: Student”或“Student”

## □消息传递

- ✓ 对象生命线间的**有向边**
- ✓ “[\*][监护条件] [返回值:=]消息名[(参数表)]”
- ✓ “\*”为迭代标记表示同一消息对同一类的多个对象发送

# 对象间的消息传递

## □同步消息

- ✓发送者等待接收者将消息处理完后再继续

## □异步消息

- ✓发送者在发送完消息后不等待接收方即继续自己的处理

## □自消息

- ✓一个对象发送给自身的消息

## □返回消息

- ✓某条消息处理已经完成，处理结果沿返回消息传回

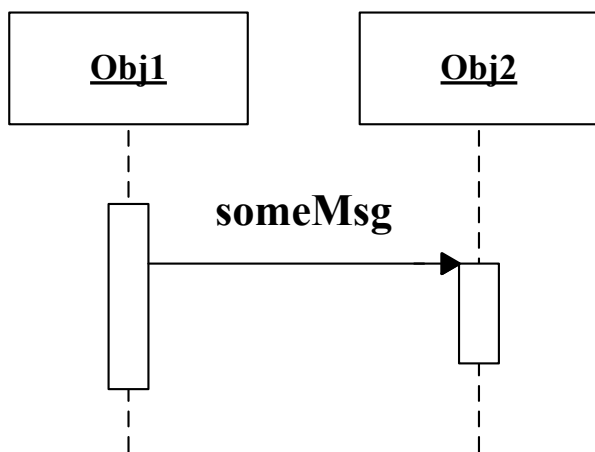
## □创建消息和销毁消息

- ✓消息传递目标对象的创建和删除

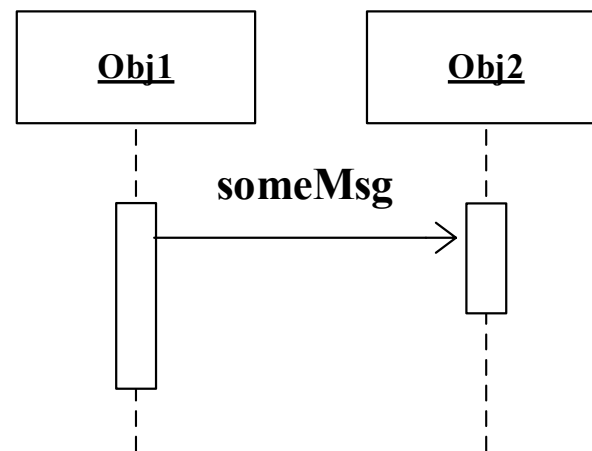
# 消息图元的表示

注意箭头的图形表示(线条和箭头形状)

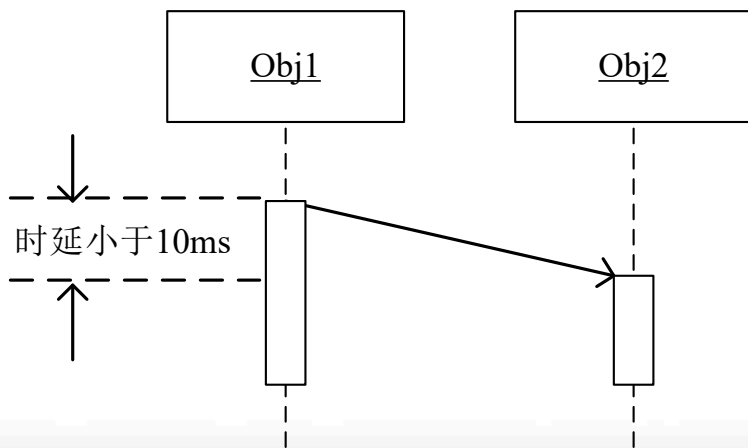
同步消息



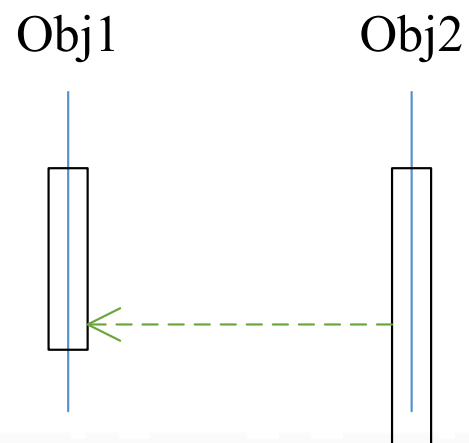
异步消息



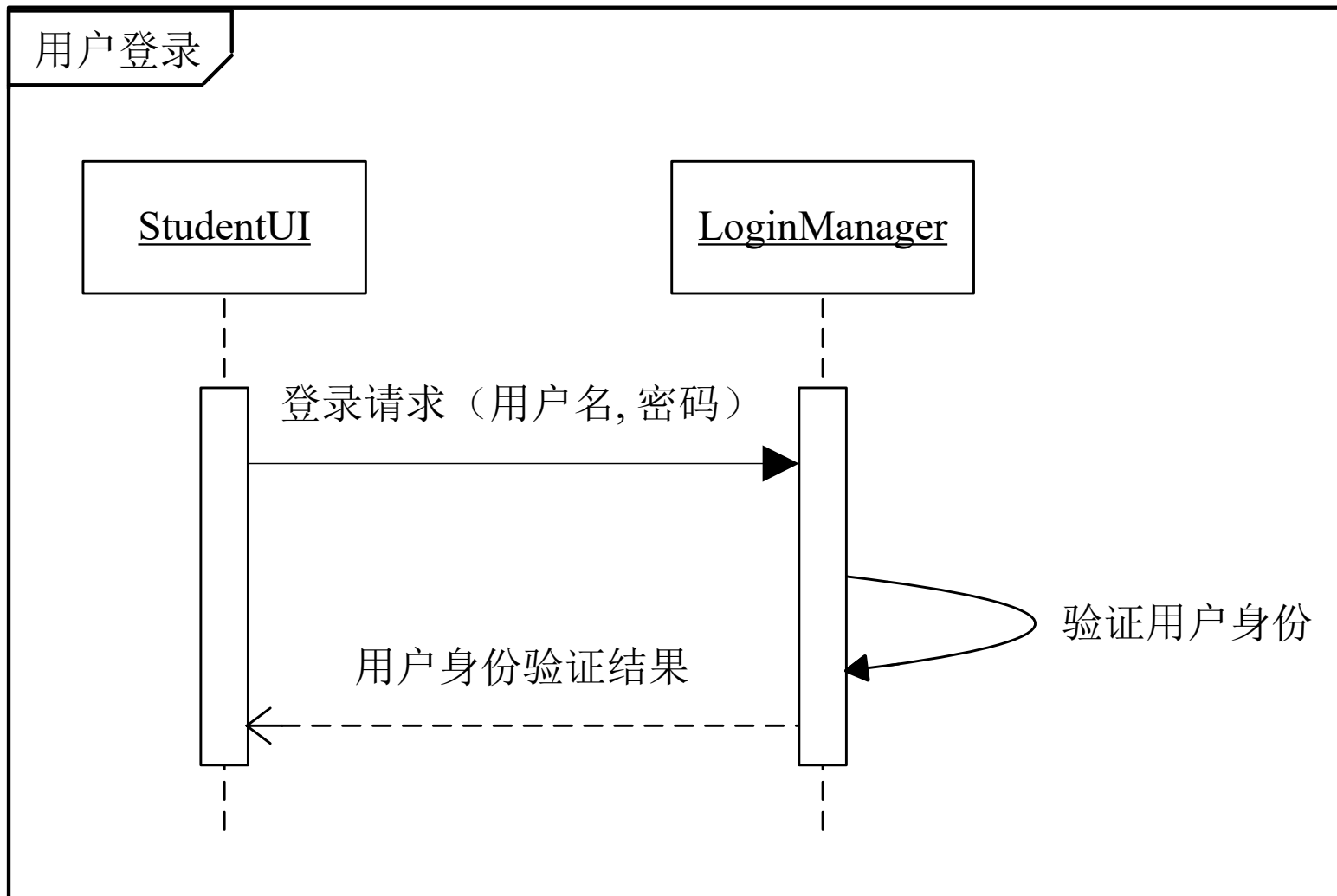
带时间  
延迟的  
消息



返回消息



# 示例：顺序图



- ✓ 对象
- ✓ 生命线
- ✓ 活跃期
- ✓ 同步消息
- ✓ 返回消息
- ✓ 自消息
- ✓ 消息名称

# 通信图的表示

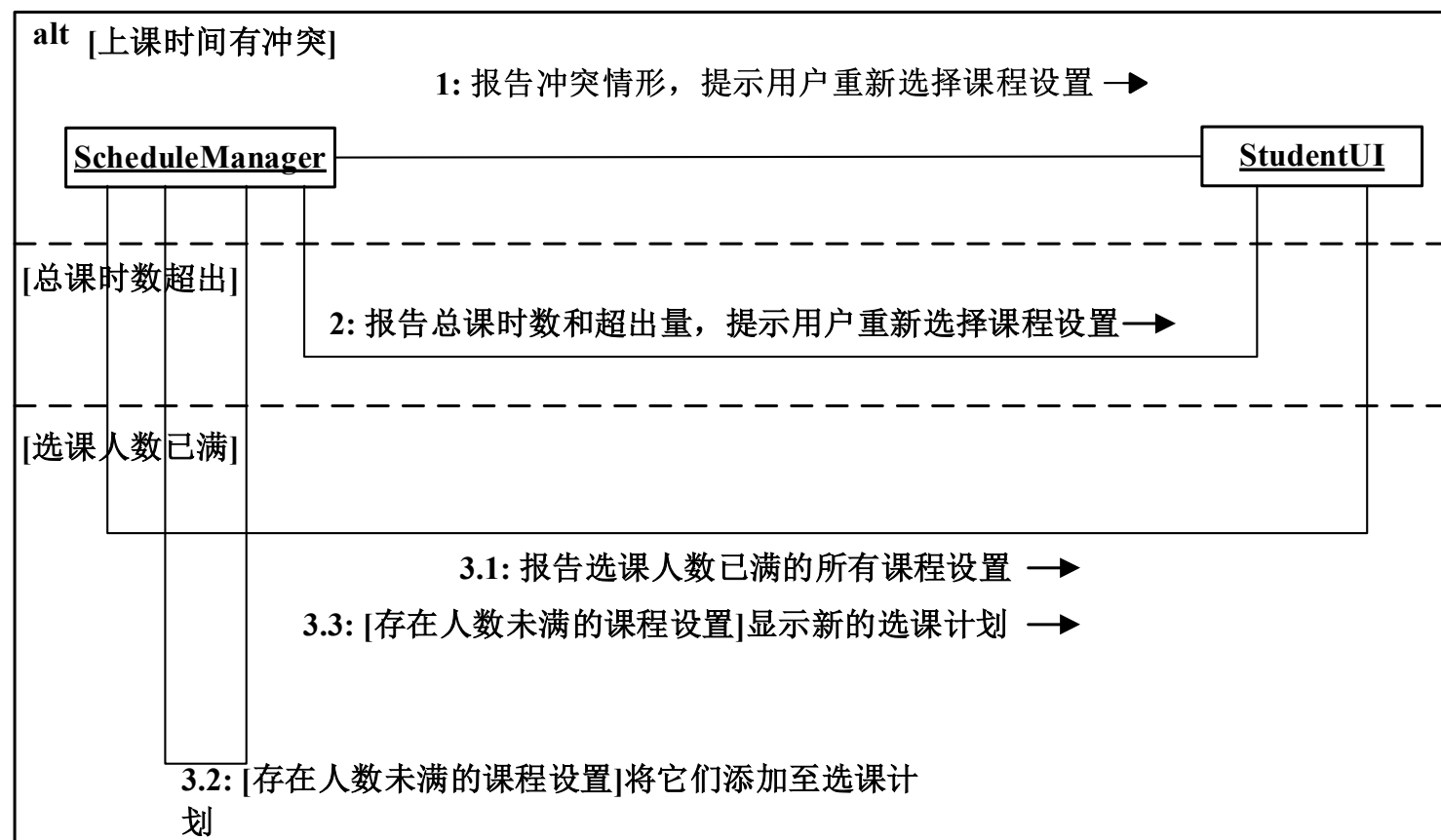
□节点表示对象

□对象间连接称为连接器

□连接器上可标示一到多条消息

□消息传递方向用靠近消息小箭头表示

□消息序号采用多层标号



# 顺序图和通信图的选取原则

## □顺序图和通信图**语义上等价**

- ✓没必要针对同一建模目标同时创建这二个图

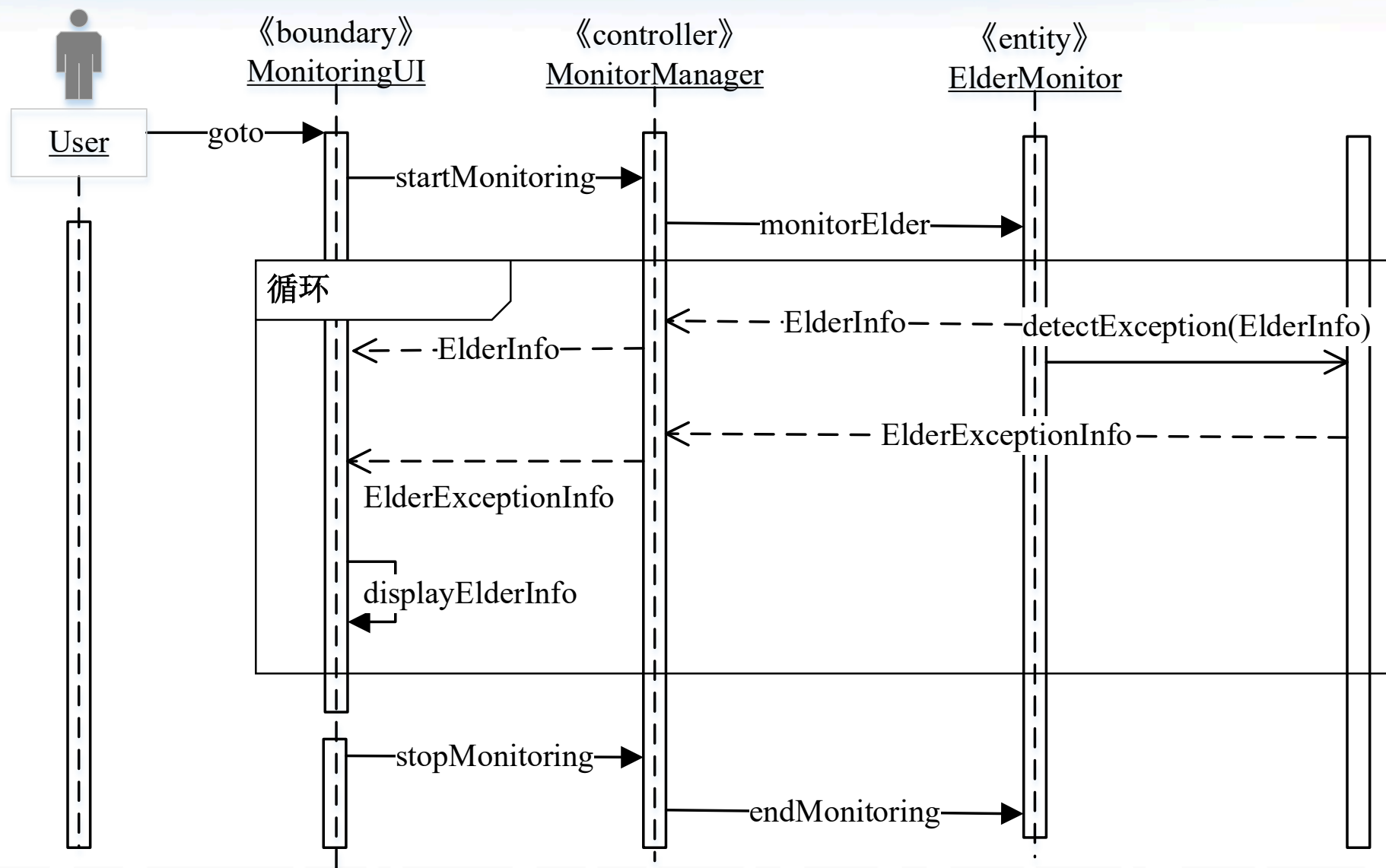
## □选择原则

- ✓当需要强调消息传递的**时间序**时采用顺序图，当需要强调对象间的**交互协作关系**时采用通信图
- ✓当刻画场景或用例的动作序列时采用顺序图，当刻画软件内部某项功能的实现构想时，采用通信图
- ✓在业务分析和需求建模阶段优先考虑顺序图；在设计和实现阶段优先考虑通信图

# 示例：顺序图

□指出顺序图中各个图形要素及其含义

□这个图是否有画的不之处



# 3 支持需求建模和分析的UML图

| 视点 | 图 (diagram)                | 说明                   |
|----|----------------------------|----------------------|
| 结构 | 包图 (package diagram)       | 从包层面描述系统的静态结构        |
|    | 类图 (class diagram)         | 从类层面描述系统的静态结构        |
|    | 对象图 (object diagram)       | 从对象层面描述系统的静态结构       |
|    | 构件图(component diagram)     | 描述系统中构件及其依赖关系        |
| 行为 | 状态图(statechart diagram )   | 描述状态的变迁              |
|    | 活动图(activity diagram)      | 描述系统活动的实施            |
|    | 通信图(communication diagram) | 描述对象间的消息传递与协作        |
|    | 顺序图(sequence diagram)      | 描述对象间的消息传递与协作        |
| 部署 | 部署图 (deployment diagram)   | 描述系统中工件在物理运行环境中的部署情况 |
| 用例 | 用例图 (use case diagram)     | 从外部用户角度描述系统功能        |